

Actuadores — Laboratorio 06

Se presentan los objetivos: comprender por qué no se puede conectar un motor DC directamente a un pin GPIO, controlar velocidad y dirección con el driver L298N mediante PWM, usar la librería Servo.h para controlar un servomotor y aplicar un potenciómetro como entrada analógica. Se explica que un motor DC requiere corrientes mucho mayores que las que puede entregar un pin de Arduino, por lo que se usa un driver como el L298N (puente H) para manejar la potencia y permitir invertir la dirección del giro.

Paso 1: Control de Motor DC

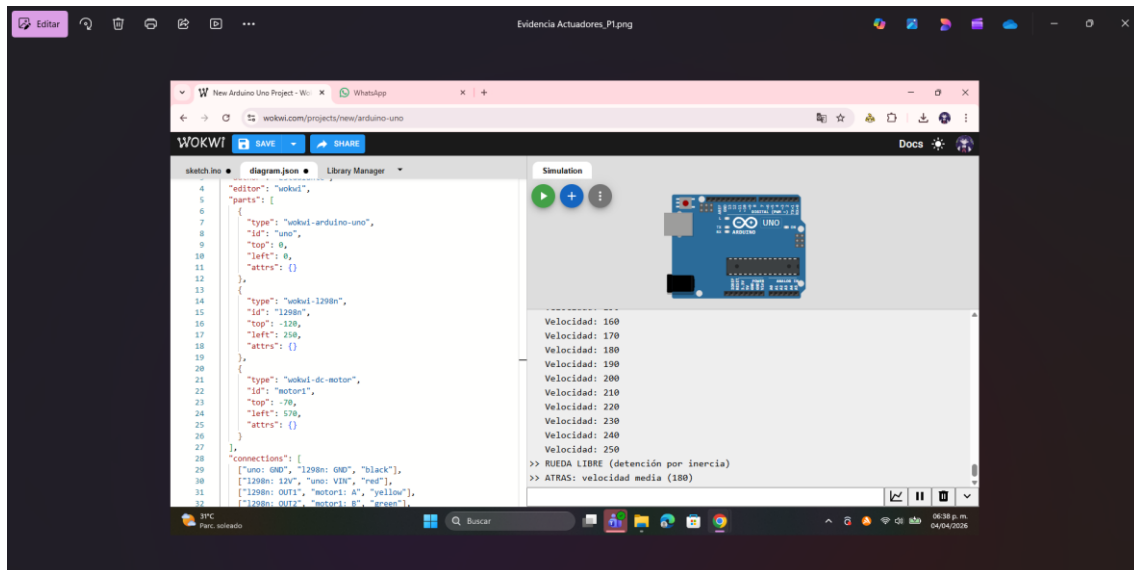
Se conectó un motor DC al driver L298N y se controló con funciones específicas:

- motorAdelante () → gira hacia adelante con velocidad variable (PWM).
- motorAtras () → gira hacia atrás.
- motorFreno () → aplica freno electromagnético.
- motorLibre () → deja el motor en rueda libre. El programa demostró cómo variar la velocidad con PWM y cómo cambiar la dirección del motor.

Se comprendió el uso de un driver para controlar motores de manera segura y eficiente.

```
sketchino
1 //
2 // LABORATORIO 06 - CONTROL DE MOTOR DC CON L298N
3 //
4
5 // DEFINICIÓN DE PINES
6 const int ENA = 10; // Pin PWM para controlar VELOCIDAD
7 const int IN1 = 8; // Pin para dirección (bit 1)
8 const int IN2 = 9; // Pin para dirección (bit 2)
9
10 //
11 // FUNCIONES DE CONTROL DEL MOTOR
12 //
13
14 // Función 1: Girar hacia ADELANTE
15 // Parámetro velocidad: 0 = detenido, 255 = máxima velocidad
16 void motorAdelante(int velocidad) {
17   digitalWrite(IN1, HIGH); // Dirección: ADELANTE
18   digitalWrite(IN2, LOW); // Dirección: ADELANTE
19   analogWrite(ENA, velocidad); // Aplica la velocidad
20 }
21
22 // Función 2: Girar hacia ATRÁS
23 void motorAtras(int velocidad) {
24   digitalWrite(IN1, LOW); // Dirección: ATRÁS
25   digitalWrite(IN2, HIGH); // Dirección: ATRÁS
26   analogWrite(ENA, velocidad);
27 }
28
29 // Función 3: FRENO electromagnético (detención brusca)
30 void motorFreno() {
31   digitalWrite(IN1, HIGH); // Ambas salidas en HIGH
32   digitalWrite(IN2, HIGH); // = cortocircuito que frena el motor
33   analogWrite(ENA, 255); // Máxima potencia para frenar más fuerte
34 }
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

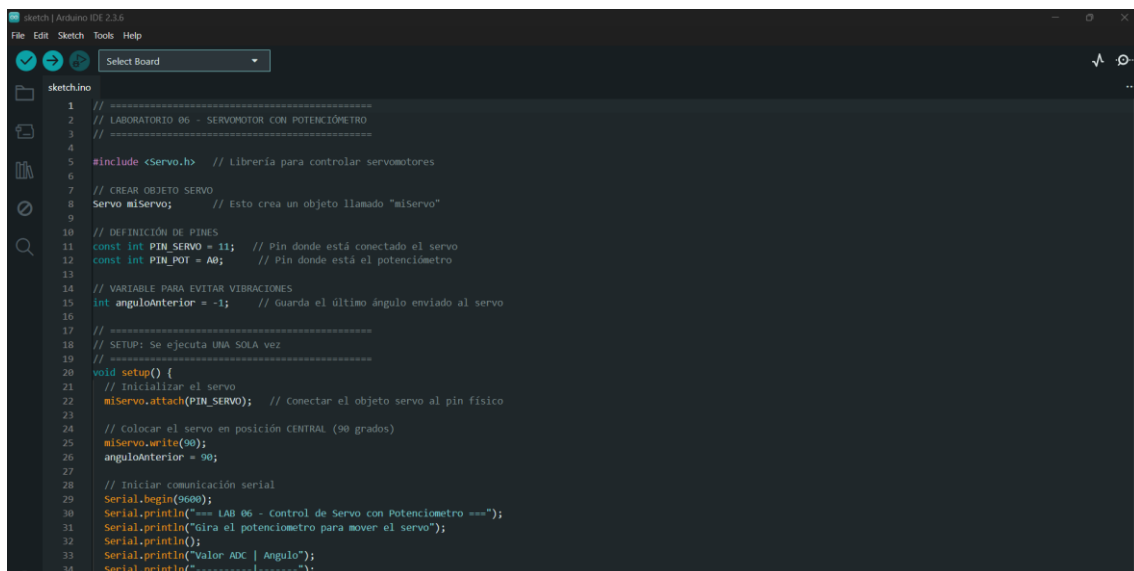
```
sketch | Arduino IDE 2.3.6
File Edit Sketch Tools Help
Select Board
sketchino
36 // Función 4: RUEDA LIBRE (se detiene por inercia)
37 void motorLibre() {
38   analogWrite(ENA, 0); // Apaga la salida del driver
39   // No importa el estado de IN1/IN2 porque ENA=0 desactiva el motor
40 }
41
42 //
43 // SETUP: Se ejecuta UNA SOLA vez al inicio
44 //
45 void setup() {
46   // Configurar los pines como SALIDA
47   pinMode(ENA, OUTPUT);
48   pinMode(IN1, OUTPUT);
49   pinMode(IN2, OUTPUT);
50
51   // Iniciar comunicación serial para ver mensajes
52   Serial.begin(9600);
53   Serial.println("== LAB 06 - Control Motor DC ==");
54 }
55
56 //
57 // LOOP: Se ejecuta infinitamente (una y otra vez)
58 //
59 void loop() {
60   // --- PRUEBA 1: Adelante con velocidad creciente ---
61   Serial.println(">> ADELANTE: velocidad creciente..");
62   for (int velocidad = 100; velocidad <= 255; velocidad += 10) {
63     motorAdelante(velocidad);
64     Serial.print("  velocidad: ");
65     Serial.println(velocidad);
66     delay(300); // Espera 300ms para ver el cambio
67   }
68
69   // --- PRUEBA 2: Rueda libre (se detiene gradualmente) ---
70   motorLibre();
71
72   // --- PRUEBA 3: Atrás con velocidad decreciente ---
73   Serial.println(">> ATRÁS: velocidad decreciente..");
74   for (int velocidad = 255; velocidad >= 100; velocidad -= 10) {
75     motorAtras(velocidad);
76     Serial.print("  velocidad: ");
77     Serial.println(velocidad);
78     delay(300); // Espera 300ms para ver el cambio
79   }
80 }
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```



Paso 2: Control de Servomotor

Se utilizó la librería Servo.h para controlar un servomotor SG90. El servo se conectó al pin 11 y se controló con un potenciómetro en A0. El valor analógico del potenciómetro se mapeó al rango de 0° a 180°, permitiendo mover el servo proporcionalmente. Se implementó un filtro para evitar vibraciones, moviendo el servo solo si el cambio de ángulo era mayor a 2°.

Se entendió cómo controlar la posición angular de un servo y cómo evitar vibraciones mediante filtrado de ruido.



```
sketch | Arduino IDE 2.3.6
File Edit Sketch Tools Help

sketch.ino
34 Serial.println("-----|-----");
35 }
36
37 // =====
38 // LOOP: Se ejecuta infinitamente
39 // =====
40 void loop() {
41 // PASO 1: Leer el valor del potenciómetro
42 // analogRead() devuelve un número entre 0 y 1023
43 int valorPot = analogRead(PIN_POT);
44
45 // PASO 2: Convertir ese valor a un ángulo (0 a 180 grados)
46 // map(valor, desde_min, desde_max, hasta_min, hasta_max)
47 int angulo = map(valorPot, 0, 1023, 0, 180);
48
49 // PASO 3: Solo mover el servo si el cambio es significativo
50 // Esto evita vibraciones por pequeñas fluctuaciones del potenciómetro
51 if (abs(angulo - anguloAnterior) >= 2) {
52 // Mover el servo al nuevo ángulo
53 miServo.write(angulo);
54
55 // Actualizar la variable del ángulo anterior
56 anguloAnterior = angulo;
57
58 // Mostrar los valores en el Monitor Serial
59 Serial.print(" ");
60 Serial.print(valorPot);
61 Serial.print(" | ");
62 Serial.println(angulo);
63 }
64
65 // Pequeña pausa para estabilizar la lectura
66 delay(20);
67 }
68 }
```

New Arduino Uno Project - Wokwi | WhatsApp

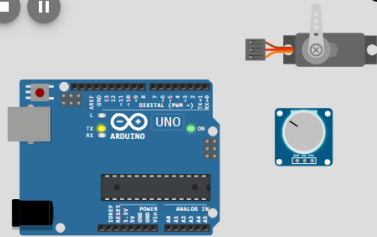
wokwi.com/projects/new/arduino-uno

WOKWI SAVE SHARE Docs

sketch.ino • diagram.json • Library Manager libraries.txt

```
20 void setup() {
21 // iniciar comunicación serial
22 Serial.begin(9600);
23 Serial.println("=== LAB 06 - Control de Servo con Potenciómetro ===");
24 Serial.println("Gira el potenciómetro para mover el servo");
25 Serial.println();
26 Serial.println("Valor ADC | Angulo");
27 Serial.println("-----|-----");
28 }
29
30 // =====
31 // LOOP: Se ejecuta infinitamente
32 // =====
33 void loop() {
34 // PASO 1: Leer el valor del potenciómetro
35 // analogRead() devuelve un número entre 0 y 1023
36 int valorPot = analogRead(PIN_POT);
37
38 // PASO 2: Convertir ese valor a un ángulo (0 a 180 grados)
39 // map(valor, desde_min, desde_max, hasta_min, hasta_max)
40 int angulo = map(valorPot, 0, 1023, 0, 180);
41
42 // PASO 3: Solo mover el servo si el cambio es significativo
43 // Esto evita vibraciones por pequeñas fluctuaciones del potenciómetro
44 if (abs(angulo - anguloAnterior) >= 2) {
45 // Mover el servo al nuevo ángulo
46 miServo.write(angulo);
47
48 // Actualizar la variable del ángulo anterior
49 anguloAnterior = angulo;
50
51 // Mostrar los valores en el Monitor Serial
52 Serial.print(" ");
53 Serial.print(valorPot);
54 Serial.print(" | ");
55 Serial.println(angulo);
56 }
57
58 // Pequeña pausa para estabilizar la lectura
59 delay(20);
60 }
61 }
```

Simulation 00:23.534 44%



597	105
638	112
575	101
567	99
589	103
616	108
647	

6. Conclusiones

En este laboratorio se aprendió a controlar actuadores básicos: motores DC y servomotores. Se comprendió la necesidad de usar drivers como el L298N para manejar corrientes altas y se aplicó PWM para variar la velocidad. Con el servomotor se exploró el control de posición angular mediante la librería Servo.h y un potenciómetro como entrada analógica. Los resultados demostraron la importancia de integrar entradas y salidas en sistemas embebidos para lograr control preciso y seguro de actuadores. En conclusión, se adquirió experiencia práctica en el manejo de motores, consolidando conocimientos esenciales para proyectos de robótica y automatización.